

RubricLoop

A rubric-building and grading agent for LLM outputs

Module 1: Design and Prototype Your AI Agent | Anurag | September 14, 2026 | v1.1

1. Summary

Teams that ship LLM features cannot tell, in a repeatable way, whether their outputs are good. They evaluate with spot checks or hand-written rubrics that drift and get applied inconsistently. RubricLoop is an agent that drafts a rubric from a task description and a few labelled examples, grades batches of outputs against it, sends a human only the cases it cannot grade with confidence, and turns the human's corrections into proposed rubric revisions. Each cycle tightens the rubric and moves the agent's grades closer to the team's own judgement.

Deliverables. This PRD, and a working prototype deployed at rubricloop.vercel.app that runs the full loop live against Claude Sonnet 4.6.

2. Problem definition

2.1 What problem are we solving?

Evaluating open-ended LLM output is a judgement task, and teams lack a way to make that judgement consistent, scalable and auditable:

- **Rubrics rot.** Criteria written at launch stop matching what 'good' means three prompt iterations later, and nobody owns updating them.
- **Grading is inconsistent and does not scale.** Two reviewers disagree 20 to 35% of the time on ordinal scales; a careful read of one output takes 1 to 2 minutes, so teams sample 20 items and hope.
- **LLM-as-judge is adopted naively.** Teams paste a rubric into a prompt and get a score with no signal about when the judge is guessing. A low-confidence grade that looks authoritative does more damage than no grade.
- **The feedback loop is missing.** When a human disagrees with an automated grade, that disagreement is lost. The rubric and the judge never improve from it.

2.2 Who is the user?

Persona	Context	Needs
Eval owner (primary)	ML engineer or applied scientist on a product team of 3 to 15 people. Owns the prompt, the eval set and the release decision.	A rubric that reflects what the team means by 'good'; grades at batch scale; a short queue of cases worth their attention; evidence for go/no-go decisions.
Product manager	Defines the feature and the bar; knows the domain, does not write eval code.	Express quality criteria in plain language; review the proposed rubric; see quality trends.
Domain reviewer	Subject-matter expert who labels	Only be asked about ambiguous cases; see

Persona	Context	Needs
	outputs. Time is scarce.	why the agent was unsure; see that labels changed the rubric.

2.3 Why is the problem worth solving?

- **It is a budgeted problem.** Every team shipping LLM features needs evaluation. Observability and eval tooling (LangSmith, Braintrust, Arize, Humanloop) is a funded category with paying buyers.
- **The cost of being wrong is high.** A regression that ships because the eval sampled $n=20$ becomes a customer-facing quality incident. A team that can measure quality iterates on prompts and models several times faster.
- **Expert time is the bottleneck.** The people qualified to judge outputs are the most expensive on the team. Cutting their review load by 80% while keeping them on the hard cases is a direct, measurable win.
- **Existing tools grade; they do not build or refine rubrics.** They assume you already have a good rubric and a labelled set. Producing those is the hard part.

2.4 Why is AI useful here?

- **The task is language-native.** Reading an output, comparing it to criteria and explaining a judgement is what LLMs do well.
- **Rubric drafting is pattern extraction from few examples.** From four good and four bad outputs a model writes the implicit criteria in under a minute; a human needs an afternoon.
- **The judgement improves with feedback.** A model can turn 'these three disagreements' into 'criterion 2 is ambiguous about tone; here is tighter wording', so each label improves the instrument as well as the score.
- **But the ground truth is human.** What counts as 'good' is a product decision the agent cannot own. RubricLoop therefore has explicit human gates, and the confidence and escalation logic is core functionality.

Agent versus prompt. A prompt can grade one output. It cannot decide which outputs need a human, ask for the label, notice that three disagreements share a cause, and propose a rubric change that fixes them. That multi-step, stateful loop is what makes this an agent.

3. Use cases

UC-1 to UC-3 are in scope for the prototype; UC-4 to UC-6 drive the P1 requirements.

Use case	Actor and trigger	Flow and outcome
UC-1 Bootstrap a rubric	Eval owner, two weeks from launch, no agreed definition of a good reply.	Writes a task description, pastes eight labelled example drafts. Agent proposes five weighted criteria with anchors, two assumptions and one question. Owner answers, reweights, approves v1. Ten minutes instead of an afternoon.

Use case	Actor and trigger	Flow and outcome
UC-2 Grade a batch after a prompt change	Eval owner; an engineer shortened replies.	Uploads 300 outputs. Agent grades each with per-criterion scores, evidence and confidence; 262 accepted, 38 queued. Aggregate shows score down 0.3, driven by 'correct resolution'. Measured answer in six minutes, 38 human reads instead of 300.
UC-3 Resolve the queue and improve the rubric	Domain reviewer; 38 items waiting.	Each card shows the tentative score and why the agent was unsure. Reviewer agrees with 29, overrides 9 with notes. Agent finds 6 share a cause, proposes a new anchor, owner approves v2, re-grade agrees on 8 of 9.
UC-4 Gate a release in CI (P1)	Platform engineer; a PR changes a prompt.	CI calls the API with the golden set and rubric version; fails the check if overall drops more than 0.2 or any criterion more than 0.5.
UC-5 Compare two models (P1)	PM deciding on a cheaper model.	Paired outputs graded plus pairwise preference; report shows win rate and least-confident pairs for spot-checking.
UC-6 Audit the grader (P1)	Eval owner asked 'how do we know the grader is right?'	Calibration view: agent-human agreement (kappa) over time, reliability curve, weakest criteria, held-out set re-graded per version.

4. The agent

4.1 Receive, decide, do, produce

Stage	Description
Receives	Task description; example outputs with optional labels and reasons; a batch of outputs; the current rubric version; human labels and override notes; the confidence threshold.
Decides	Which criteria matter and their weights; anchor wording so two graders would agree; per-criterion scores; confidence per grade and therefore results table or review queue; which disagreements share a cause; what rubric edit resolves them; whether to ask a human a question.
Does	Drafts and revises rubrics; grades outputs concurrently; extracts evidence quotes; routes items to the queue; clusters disagreements; proposes rubric diffs; re-grades affected items after an approved change; records a trace of every step.
Produces	Versioned rubric (criteria, weights, anchors, evidence hints, assumptions, questions); per-item grade records (scores, rationale, evidence, confidence, uncertainty reason, flags); aggregate report; review queue; proposed edits with justification; agent trace.

4.2 The loop

1. **Draft.** Produce rubric v(n) with assumptions and questions. Human approves or edits (gate 1).

2. **Grade.** Score every criterion, cite evidence, estimate confidence. Items below threshold or carrying a risk flag go to the queue with a stated reason.
3. **Ask.** A human agrees or overrides each queued item with a note (gate 2). Nothing in the queue counts as graded until a human acts.
4. **Refine.** Cluster disagreements, propose a diff, re-grade the affected items, report the agreement change. Human approves each edit, creating v(n+1) (gate 3).

4.3 Autonomy and human intervention

The agent has full autonomy over anything that can be undone or checked, and none over anything that defines the standard or that another system consumes as truth.

Decision or action	Who decides	Why
Draft rubric criteria, weights, anchors	Agent proposes, human approves	The rubric is the definition of quality; a person must own it.
Grade above the confidence threshold	Agent	Reversible and auditable; a sample is re-checked by a human on every job.
Grade below threshold or flagged	Agent tentatively; human labels	An unflagged guess is the failure this product exists to remove.
Set the confidence threshold	Human	Trades reviewer time against grade quality.
Route an item to the queue	Agent	Asking cannot cause harm; over-asking costs reviewer time and is tracked.
Change the rubric after disagreements	Agent proposes diff, human approves each edit	A rubric change alters every future grade; it needs a named approver and a version bump.
Re-grade after an approved change	Agent	Deterministic consequence of an approved decision; shown as a delta.
Pass/fail a release in CI (P1)	Agent computes; thresholds and override are human	Blocking a merge is consequential; policy must be human-set.

Invariants. The agent never edits an approved rubric in place, never reports grades against an unapproved version, and never silences a low-confidence grade. These are enforced in code, not in the prompt.

5. System map

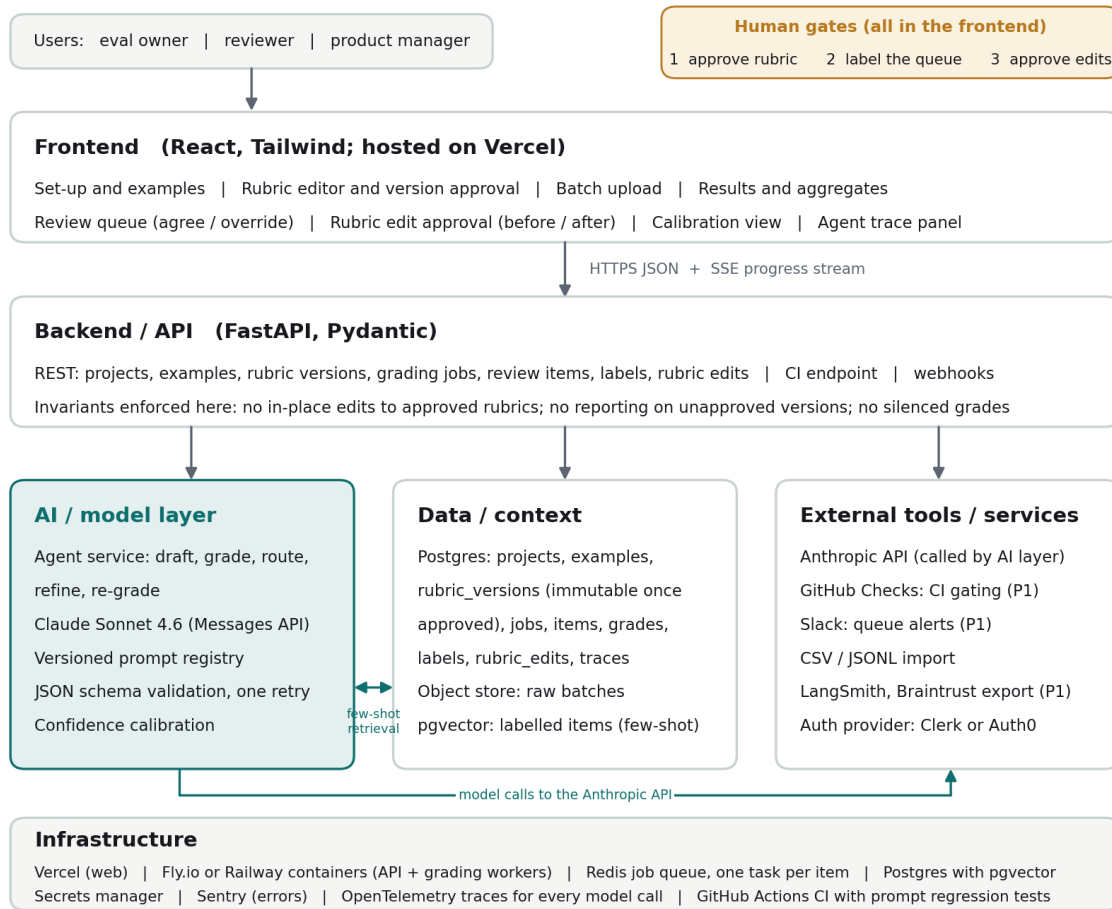


Figure 1. System map. Solid arrows are request paths; teal arrows are the AI layer's calls to data and to the model API.

Layer	Components and responsibilities
Frontend	React + TypeScript, Tailwind. Rubric editor with weights and anchors; batch upload (paste, CSV, JSONL); results table with per-criterion breakdown; review queue with keyboard labelling; edit approval with before/after; calibration view; agent trace panel.
Backend / API	FastAPI, Pydantic. REST for projects, rubric versions, jobs, review items, labels; enforces the three invariants; streams job progress; CI endpoint; webhooks.
AI / model layer	Agent service calling Claude Sonnet 4.6 (Opus-class optional for draft and refine). Versioned prompts; JSON schema validation with one retry; confidence estimation and routing; disagreement clustering; diff generation; re-grading.
Data / context	Postgres: projects, examples, rubric_versions (immutable once approved), grading_jobs, items, grades, labels, rubric_edits, traces. Object store for raw uploads. pgvector index of labelled items supplies the grader with the most similar past labels as few-shot context.
Infrastructure	Vercel for the web app; Fly.io or Railway containers for API and workers; Redis queue (one

Layer	Components and responsibilities
	task per item, retry with backoff); managed Postgres with pgvector; secrets manager; Sentry; OpenTelemetry traces for every model call.
External tools	Anthropic API; GitHub Checks for CI gating (P1); Slack notifications when the queue exceeds a threshold (P1); import from and export to LangSmith and Braintrust dataset formats (P1).

6. Product specification

6.1 Goals

1. Cut human grading effort by 80%: on a 300-item batch, at most 60 items need a human label, with agent-human agreement of at least 85% on the rest.
2. Time to first approved rubric under 15 minutes from project creation.
3. Every rubric change is versioned, justified and approved by a named person.
4. Agent confidence is calibrated: items graded at confidence 0.8 or higher agree with humans at least 80% of the time.
5. Business: three design-partner teams paying within 90 days of private beta, at least one using the CI endpoint.

6.2 Non-goals

- Fine-tuning a custom judge model (few-shot retrieval captures most of the gain; revisit if agreement plateaus).
- Generating the outputs being evaluated (RubricLoop grades; it does not run the customer's prompts).
- Automated pass/fail decisions with no human-set policy.
- Multi-modal outputs in v1; replacing existing observability tools (integrate instead).

6.3 User stories

- As an eval owner, I want to describe my task and paste example outputs so that the agent drafts a rubric without me writing criteria from scratch.
- As an eval owner, I want to see the agent's assumptions and questions so that I can correct misunderstandings before any grading happens.
- As an eval owner, I want to edit and approve a rubric version so that the team grades against a standard I own.
- As an eval owner, I want per-criterion scores with evidence so that I can see which criterion moved when quality changed.
- As an eval owner, I want to set the confidence threshold so that I control how much human review the team spends.
- As a reviewer, I want to see only items the agent was unsure about, and why, so that my time goes to cases that matter.

- As a reviewer, I want to agree or override with a one-line note so that I can clear 40 items in 20 minutes.
- As an eval owner, I want to see proposed rubric edits with the disagreements that motivated them so that I can approve or reject each one.
- As a platform engineer, I want an API that grades a dataset against thresholds I set so that I can gate merges in CI.

6.4 Requirements

Must-have (P0), validated by the prototype

ID	Requirement	Acceptance criteria
P0-1	Rubric drafting	Given a task description and 0 or more examples, within 30 s a rubric of 3 to 7 criteria appears with name, description, weight (sum 100), anchors for 1, 3, 5, evidence hint, plus assumptions and questions. Invalid JSON is retried once, then surfaced.
P0-2	Rubric editing and versioning	Any field is editable; weights must sum to 100; approving creates an immutable, numbered version. Approved versions cannot be edited in place.
P0-3	Batch grading	Each output gets per-criterion scores, weighted overall, a rationale under 40 words, a verbatim evidence quote, confidence in [0,1], and an uncertainty reason when below threshold. Runs concurrently with progress; failures retry once and do not block others.
P0-4	Confidence routing	Items below the threshold (default 0.7, adjustable) or flagged policy_risk / injection_suspect enter the queue and are excluded from aggregates until labelled.
P0-5	Human labelling	Reviewer agrees or overrides (new overall score plus note). Agreement rate is displayed.
P0-6	Rubric refinement	Given at least one override, the agent proposes 1 to 4 edits (reword, add criterion, reweight), each citing disagreement IDs. Approving any creates a new version and re-grades the overridden items, showing before/after agreement.
P0-7	Agent trace	Every draft, grade, route, ask, refine and re-grade step is logged with summary and decision and shown in a panel.
P0-8	Aggregate report	Mean overall, per-criterion means, counts accepted, queued and labelled, agreement rate.

Nice-to-have (P1) and later (P2)

- **P1:** run comparison and pairwise preference (UC-5); CI endpoint and GitHub Check (UC-4); few-shot retrieval of labelled items (target +5 points agreement); calibration dashboard with kappa and reliability diagram (UC-6); Slack alerts; CSV/JSONL import and LangSmith/Braintrust export.

- **P2:** multi-modal items; multi-reviewer labelling; active learning to pick which items to label; rubric templates; fine-tuned judge once a project has 2,000 or more labels.

6.5 Success metrics

Metric	Type	Target	Measured by
Time to first approved rubric	Leading	< 15 min	project.created to first approval
Share of items routed to review	Leading	20% or less at default threshold	queued / items per job
Agent-human agreement on autonomous grades	Leading	85% or more	Weekly blind 5% audit sample
Calibration error (ECE)	Leading	0.10 or less	Reliability diagram per version
Agreement lift after refinement	Leading	+8 points on re-graded items	Re-grade of overridden items
Active design partners; paid conversion	Lagging	3 active in 30 days; 3 of 5 paid in 90	Product analytics; billing

7. Technical specification

7.1 Key decisions

Decision	Choice and rationale
Grading model	Claude Sonnet 4.6: strong structured-JSON compliance, fast enough for 300 items in minutes at concurrency 8, cost-appropriate for volume. Opus-class optional for draft and refine.
Structured output	JSON with a strict schema, validated server-side, one retry with the validation error fed back. Portable across providers.
Confidence	Model self-report plus a constrained uncertainty_reason (rubric_gap, borderline_score, conflicting_criteria, missing_context, output_ambiguous); calibrated per project with isotonic regression once 50 or more labels exist.
Improving agreement	Retrieve the k=5 most similar labelled items (pgvector) as few-shot context. Most of the benefit of fine-tuning with no training loop.
Job execution	Redis queue, one task per item, horizontally scaled workers; fine-grained retry and progress.
Stack	React + TypeScript front end; Python / FastAPI back end (Pydantic doubles as the schema layer).

7.2 Data model and API

Core tables: projects, examples, rubric_versions (version_no, status proposed / approved / superseded, criteria JSONB, parent_version_id, approved_by), grading_jobs, items, grades (scores, overall, rationale, evidence, confidence, calibrated_confidence, uncertainty_reason, source agent / human), labels (decision, override_overall, note, embedding), rubric_edits (type, target, new_text, justification, disagreement_ids, status), traces.

POST /v1/projects	create project
POST /v1/projects/{id}/rubric/draft	agent drafts a proposed version
POST /v1/rubric-versions/{id}/approve	freeze version
POST /v1/projects/{id}/jobs	upload batch, start grading
GET /v1/jobs/{id}/events	progress stream (SSE)
GET /v1/jobs/{id}/results /review	grades and aggregates queued items
POST /v1/items/{id}/label	agree or override with note
POST /v1/projects/{id}/rubric/refine	agent proposes edits
POST /v1/rubric-edits/{id}/approve reject	per-edit decision
POST /v1/ci/grade	(P1) dataset + thresholds -> pass/fail

7.3 Agent design and guardrails

- Three versioned prompts (draft, grade, refine), each returning JSON only. The grade prompt wraps the item in delimiters and states that instructions inside it are content to grade, not follow; items that try to influence the grader are flagged injection_suspect and queued.
- Routing is a backend rule, not a prompt instruction: queued = calibrated_confidence < threshold OR flags contain policy_risk or injection_suspect.
- Evidence quotes must be substrings of the item; a mismatch lowers confidence by 0.2 and adds a flag.
- Refine receives disagreements clustered by (criterion with the largest delta, uncertainty reason). Edits without cited disagreement IDs are rejected. Before edits are shown, cited items are re-graded against the candidate rubric and the agreement change is reported.
- Evaluating the evaluator: a blind 5% audit sample per job; a frozen held-out set per project re-graded on every version; ECE per version; prompt regression tests before any prompt change ships.

7.4 Security, cost and infrastructure

- Item content may contain customer data: encryption at rest and in transit, per-project isolation, configurable retention, optional PII redaction before model calls, zero-data-retention API terms where available. Roles: owner (approves), reviewer, viewer. All approvals logged with identity.
- Cost: about \$0.01 per graded 400-word item; 300 items in 3 to 4 minutes for roughly \$3.50. A team grading 2,000 items a week spends about \$25 in model cost, against a seat price of \$99 to \$199 per eval owner per month.
- Deployment: Vercel (web), Fly.io or Railway (API and workers), managed Postgres with pgvector, Upstash Redis, GitHub Actions CI with prompt regression tests, Sentry and OpenTelemetry.

7.5 Harness

Three layers of checks surround the agent. The first two exist in the prototype; the third is Phase 1.

Layer	What it does	Status
Runtime harness	Wraps every model call: JSON schema validation with one retry; evidence must be a verbatim substring or confidence drops 0.2 and a flag is added; injection detection; routing to review is a code rule, not a prompt instruction; a trace records every step.	In prototype (src/lib.js)
Evaluation harness	Grades a frozen golden set (12 outputs with human per-criterion scores) using the real grade prompt and reports agreement within 0.5, mean absolute error per criterion, agreement by confidence bucket, items routed to review, and recall of expected risk flags. Exits non-zero below a target so it can gate prompt changes in CI.	In prototype (harness/run.mjs)
Unit tests	Cover the shared pure functions: weight normalisation, weighted overall, routing rule, grade post-processing, edit application, injection regex. Writing them caught a real defect: a model score of 0 was silently replaced by the default 3 instead of being clamped to 1.	In prototype (harness/test.mjs)
Production harness	Timeouts, rate limits, per-job token budget, server-side enforcement of the three invariants, blind 5% audit sample per job, held-out set re-graded on every rubric version, calibration fitting.	Phase 1

8. Prototype

The prototype is a React application deployed on Vercel with a serverless proxy that keeps the API key server-side. It runs the full loop against Claude Sonnet 4.6 and validates P0-1 to P0-8: load a sample project, draft and approve rubric v1, grade the batch, label the queue, approve proposed edits to create v2, and see the before/after agreement on re-graded items. A side panel shows the agent trace with each human gate.

Five sample projects are built in, each with labelled examples and a batch seeded with known failure modes, so the agent can be watched on tasks with different kinds of difficulty:

Sample project	What it stresses	Seeded failure modes in the batch
Customer support replies (furniture store)	Policy rules and tone	Refund outside the 30-day window, promised delivery date, return demand on damaged goods, template non-answer, grader-directed instruction.
SQL query explanations for analysts	Technical accuracy where a fluent wrong answer is the risk	LEFT JOIN semantics misread, HAVING filter omitted, window function described as GROUP BY, NOT IN with NULL trap missed, jargon for a non-technical reader, grader-directed instruction.
Blameless incident postmortem summaries	Structural completeness plus a hard rule	Numbers changed from the notes, an engineer named and blamed, missing timeline and owners, grader-directed instruction.
Earnings press-release	Faithfulness to a source	Computed figure not in the source, invented prior-

Sample project	What it stresses	Seeded failure modes in the batch
summaries		year comparison, a near-miss number (\$3.2B for \$3.02B), investment advice, forward-looking claim not in the source, grader-directed instruction.
Metric definitions for a data catalog	Faithfulness to code: every claim must be supported by the SQL	Formula contradicted (gross described as net, discounts added), grain missing, date filter dropped, plan-at-query-time described as plan-at-signup, invented owner and refresh schedule, grader-directed instruction.

The repository also contains the harness described in 7.5: a reference rubric, a golden set with human scores, an evaluation script that reports agreement and calibration, and unit tests for the shared scoring and routing code. The app and the harness import the same prompt and the same functions, so the harness measures the code that ships.

Deliberate limitations: state is in memory; no backend, auth or persistence; confidence is the model's self-report (calibration needs 50 or more labels); disagreement clustering is done by the model rather than a deterministic pre-pass; batch size is capped at 25.

9. Risks, open questions and phasing

Risk	Mitigation
Confidence is poorly calibrated, so the queue misses the wrong items	Post-hoc calibration; blind audit sample; conservative default threshold; uncertainty reasons make miscalibration diagnosable.
Refinement over-fits to a few disagreements	Re-grade against the held-out set before showing an edit; warn when agreement does not improve; every edit human-approved.
Users treat agent grades as ground truth and stop reviewing	Audit sample is mandatory; trust metrics on the dashboard; the queue can be narrowed but not disabled.
Incumbent eval platforms add rubric generation	Move fast on the refinement loop and calibration; integrate with incumbents rather than compete on observability.

Open questions. Default scale 1 to 5 or pass/fail per criterion (product; support both). Confidence threshold that minimises reviewer effort at 85% agreement (data; resolve in beta). Whether design partners require a VPC or self-hosted model option (sales and legal; blocking for beta).

Phasing. Phase 0, prototype (done). Phase 1, private alpha, 4 weeks: backend, Postgres, auth, job queue, invariants server-side. Phase 2, design-partner beta, 6 weeks: few-shot retrieval, calibration, run comparison, Slack alerts. Phase 3, 4 weeks: CI endpoint and GitHub Check, exports, billing.